
FSM 과 행동트리를 활용한 시스템 AI 성능비교

제출일 : 2025 년 6 월 3 일

지도교수 : 박 상 원 교수님

정보통신공학과

202104593

안재현

정보통신공학과

202104224

서민주

대 학 : 공 과 대 학

학 과 : 정보통신공학과

학 번 : 202104593
202104224

성 명 : 안재현
서민주

이 논문을 제출 하오니 승인하여 주십시오.

2025 년 6 월 3 일

성 명 : 안재현 (인 **안재현**)

서민주 (인 **서민주**)

위 학생의 논문 제출을 승인함.

2025 년 6 월 3 일

지 도 교 수 : 박상원 (인)

그림 목차 및 표 목차

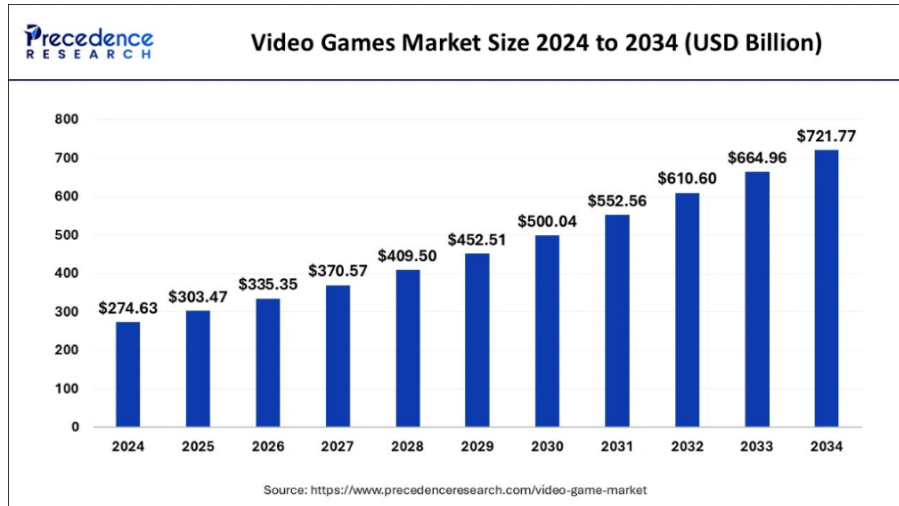
[그림 1- 1] 게임 시장 성장 예측 그래프.....	6 쪽
[그림 1 - 1] FSM 모델 구조	7 쪽
[그림 1 - 1] 행동 트리 기반구조	7 쪽
[그림 2- 1] Unreal engine 에서의 bt 구조 사용.....	10 쪽
[그림 2 -1] Unity engine 에서 Nodecanvas 를 사용한 bt 구조.....	10 쪽
[그림 2 - 3] 『로열 블러드』에서 실제 도입된 행동 트리 로직 (BT 구조).....	12 쪽
[그림 2 - 3] 『로열 블러드』 이벤트 기능에서 실제 도입된 행동 트리 로직 (BT 구조).....	13 쪽
[그림 2 - 3] 『로열 블러드』에서 공개한 BT 구조를 도입했을때의 최종 결과.....	13 쪽
[그림 2 - 4] unity 에서 사용가능한 BT 종류.....	15 쪽
[그림 3 - 2] 몬스터 AI FSM 흐름도.....	16 쪽
[그림 3 - 2] 무한맵 AI FSM 흐름도.....	17 쪽
[그림 3 - 2] 웨이브 AI FSM 흐름도.....	17 쪽
[그림 3 - 2] 플레이어 무기 전환 AI FSM 흐름도.....	18 쪽
[그림 3 - 3] 몬스터 AI BT 흐름도.....	18 쪽
[그림 3 - 3] 무한맵 AI BT 흐름도.....	19 쪽
[그림 3 - 3] 플레이어 무기전환 AI BT 흐름도.....	19 쪽
[그림 3 - 3] 웨이브 AI BT 흐름도	20 쪽
[그림 3 - 4] 정량적 실험 비교항목.....	21 쪽
[그림 3 - 4] 정성적 실험 비교항목.....	21 쪽

목차

1. 서론	
1.1 연구 배경.....	5 쪽
1.2 연구 목표.....	6 쪽
1.3 연구의 범위 및 논문 구성.....	7 쪽
2. 기존 연구 및 관련 기술	
2.1 게임 ai 기술의 발전 현황.....	8 쪽
2.2 FSM (Finite State Machine) 개요 및 적용 사례.....	9 쪽
2.3 Behavior Tree 개요 및 적용 사례.....	11 쪽
2.4 BT(Behavior Tree) 구조 도구의 등장과 확산.....	12 쪽
3. 시스템 아키텍처 및 구현 방법	
3.1 개발 환경 및 방법론.....	14 쪽
3.2 몬스터 FSM 구조 구현.....	15 쪽
3.3 몬스터 BT 기반 구조 구현.....	17 쪽
3.4 실험 시나리오 및 테스트 항목 정의.....	19 쪽
4. 실험 결과 및 분석	
4.1 정량적 분석 결과 (반응 속도, 전환 효율 등).....	20 쪽
4.2 정성적 분석 결과 (몰입도, 자연스러움, 설문 응답 등).....	21 쪽
4.3 FSM 과 BT 의 성능 비교 종합.....	22 쪽
5. 결론	
5.1 연구 요약 및 결론.....	23 쪽
5.2 향후 연구 방향.....	24 쪽
6. 참고 문헌	24 쪽

제 1 장 서론

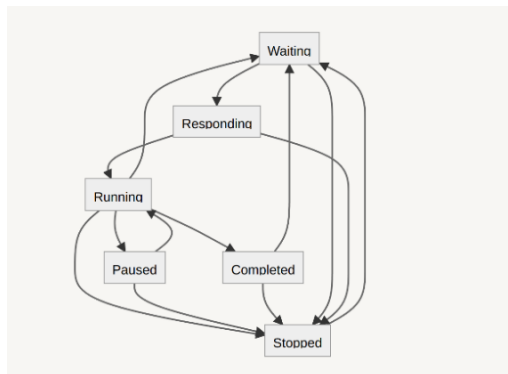
1.1 연구배경 및 목적



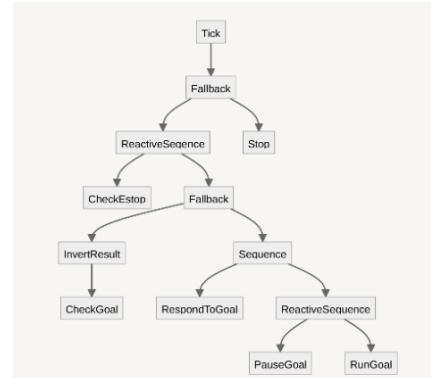
게임 산업은 기술의 발전과 함께 급격한 성장을 이루고 있으며, 그와 더불어 게임 콘텐츠의 복잡성과 정교함 역시 빠르게 증가하고 있다. 과거에는 소규모 인력과 제한된 예산으로 제작되던 게임들이, 최근에는 수백억 원 이상의 개발 비용과 수백 명 규모의 인력이 투입되는 대형 프로젝트로 진화하고 있다. 이처럼 게임의 규모가 대형화되고 시스템이 복잡해지면서, 게임 내부의 인공지능(AI) 알고리즘 또한 보다 정교한 설계와 구조적 유연성을 요구받고 있다.

초창기 게임 개발에서는 Finite State Machine(FSM) 방식이 주된 알고리즘 구조로 사용되었다. FSM은 상태를 명확히 구분하고 조건에 따라 상태를 전이시키는 구조로, 직관적이고 구현이 간단하며 상태를 시각적으로 파악하기 용이하다는 장점을 지닌다. 이러한 구조는 적은 수의 상태와 단순한 전이 로직을 기반으로 하는 게임 시스템, 예를 들어 단일 행동 단위의 NPC, 아이템 상호작용, 간단한 게임 룰 처리 등에서 매우 효과적이었다.

그러나 현대 게임은 캐릭터, 몬스터, NPC, UI 시스템, 이벤트 처리 등 다양한 요소들이 유기적으로 작동하며 복잡한 상호작용을 요구하고 있다. 이로 인해 FSM의 구조적 한계가 명확히 드러나기 시작했다. 상태 수가 많아질수록 전이 조건이 기하급수적으로 늘어나며 로직이 복잡해지고, 새로운 기능 추가 시 기존 구조 전체에 영향을 미치게 되어 유지보수가 어려워진다. 또한 조건 기반 전이 방식은 상황에 따른 유연한 대응이나 전략적 판단을 반영하기 어려운 단점이 있다.



FSM 구조



BT 구조

이러한 한계를 극복하기 위한 대안으로 Behavior Tree(BT) 구조가 주목받고 있다. BT는 행동을 트리 구조로 설계하여, 조건에 따라 계층적으로 행동을 분기시키는 방식으로 동작한다. 이 구조는 모듈화와 재사용성이 뛰어나며, 행동 우선순위 조정이나 조건부 행동 삽입이 용이하다는 점에서 복잡한 게임 로직 구현에 적합하다. 특히 행동의 흐름이 명확히 드러나고 유지보수가 용이하다는 점에서, 현재의 복잡하고 변화가 잦은 게임 개발 환경에 매우 효과적인 방식으로 평가받고 있다.

이러한 변화는 특정 캐릭터나 몬스터의 행동 알고리즘에만 국한되지 않는다. 게임 전반의 다양한 AI 및 시스템 설계에 있어서 FSM에서 BT로의 전환이 활발히 이루어지고 있으며, 전략적 판단이 필요한 상황, 플레이어 반응 기반 동작, 협동 AI, 패턴 변화, UI 상태 제어 등 다양한 분야에서 BT 구조가 도입되고 있다. 이는 게임 개발의 효율성과 확장성을 동시에 만족시키려는 업계 전반의 흐름이라 볼 수 있다.

본 연구는 이러한 배경에서, FSM과 BT 구조의 차이를 실험적으로 분석하고, 게임 내 다양한 AI 구조에 적용될 수 있는 실용적 설계 기준을 탐색하는 데 목적이 있다.

1.2 연구목표

본 연구의 주요 목적은 현재 게임 산업 전반에서 빠르게 변화하고 있는 인공지능 알고리즘 구조의 흐름을 반영하여, 기존의 FSM(Finite State Machine) 기반 구조와 최근 주목받고 있는 Behavior Tree(BT) 기반 구조의 차이를 실험적으로 비교·분석하는 것이다. 구체적으로는 Unity 엔진을 활용한 게임 환경 내에서 동일한 조건과 시나리오를 설정한 뒤, 각각 FSM과 BT 구조를 적용한 AI 시스템을 구현하고, 이들 간의 성능 차이를 다각도에서 평가하는 과정을 중심으로 한다.

FSM과 BT는 게임 알고리즘 설계 방식에서 서로 다른 구조적 철학과 기술적 특성을 가지고 있으며, 이러한 차이는 AI의 반응 방식, 전략적 다양성, 유지보수의 용이성 등에 직접적인 영향을 미친다. 따라서 본 연구는 각 구조가 실제 게임 시스템 내에서 어떠한 방식으로 행동을 전개하고, 어떠한 조건에서 강점과 약점을 보이는지를 분석하고자 한다. 이를 위해 플레이어의 거리나 상태, 주변 환경 조건 등을 변수로 삼아 상황 인식 기반의 행동 조건을 설정하고, 두 구조가 이를 어떻게 처리하는지를 정량적 지표(예: 반응 속도, 행동 전환 시간, 오류율 등)와 정성적 지표(예: 몰입감, 행동의 자연스러움, 예측 가능성 등)를 통해 종합적으로 비교할 예정이다.

또한 단순한 알고리즘 구현에 그치지 않고, 실험 결과를 기반으로 실질적인 게임 개발 현장에서 적용 가능한 AI 설계의 기준과 방향을 제시하는 것 또한 본 연구의 중요한 목표 중 하나이다. 특히 향후 다양한 게임 장르와 규모에 따라 요구되는 AI 시스템의 복잡성과 확장성을 고려할 때, 어떤 구조가 더 적합한지를 판단할 수 있는 실증적 근거를 마련하는 데 의의가 있다. 궁극적으로 본 연구는 게임 AI 설계에 있어 구조 선택의 합리성을 높이고, 더 나아가 게임 개발의 효율성과 플레이어 경험의 질을 동시에 향상시키는 데 기여하고자 한다.

1.3 연구의 범위 및 논문 구성

본 논문은 Unity 기반의 게임 환경에서 FSM(Finite State Machine)과 Behavior Tree(BT) 알고리즘을 각각 적용한 몬스터 AI를 구현하고, 동일한 조건에서 두 구조의 성능을 정량적·정성적으로 비교·분석하는 데 그 범위를 둔다. 비교 항목은 반응 속도, 오류율, 행동 전환 효율성, 행동 다양성과 같은 정량 지표뿐 아니라 몰입도, 자연스러움, 유지보수성과 같은 주관적 체감 요소도 포함된다. 이를 통해 각 구조의 장단점을 파악하고 실제 게임 개발 현장에서의 활용 가능성과 효율적인 AI 설계 방향을 제시하고자 한다. 본 논문은 총 5장으로 구성되며, 2장에서는 게임 AI 기술의 이론적 배경과 관련 선행연구를 검토하고, 3장에서는 실험에 사용된 게임의 개발 환경과 AI 구현 방법을 설명한다. 4장에서는 실험 결과를 바탕으로 FSM과 BT 구조의 성능을 다각도로 비교·분석하고, 마지막 5장에서는 연구 결과를 요약하고 향후 연구 방향을 제안한다.

제 2 장 기존 연구 및 관련 기술

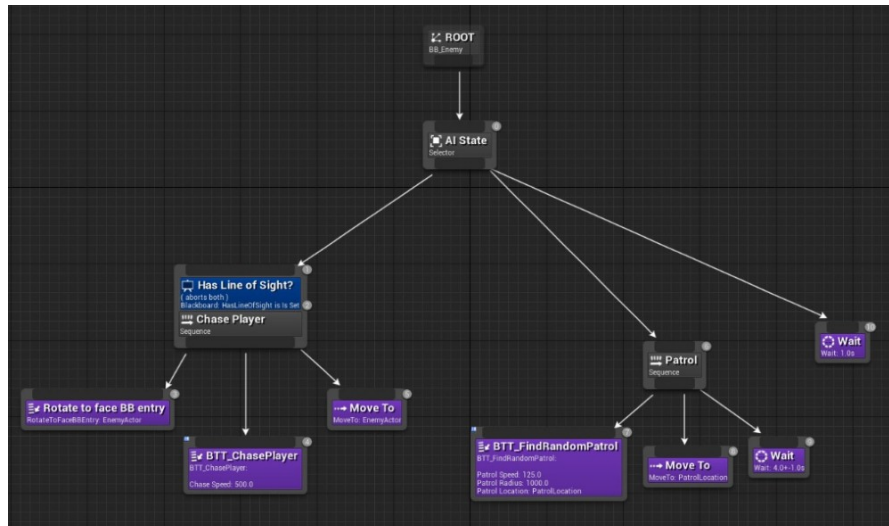
2.1 게임 AI 기술의 발전 현황

게임 인공지능(AI)은 디지털 게임의 진화와 함께 지속적으로 발전해 왔다. 1980 년대와 1990 년대 초기에는 간단한 규칙 기반(Rule-Based) 시스템이 주류를 이루었으며, 이는 if-else 조건문으로 구성된 단순한 반응형 구조였다. 당시 AI는 정해진 루틴을 반복 수행하며, 플레이어의 입력에 고정된 방식으로 반응하는 수준이었다. 그러나 게임 환경이 복잡해지고 플레이어의 기대 수준이 높아짐에 따라, 보다 정교하고 유연한 AI 시스템의 필요성이 대두되었다.

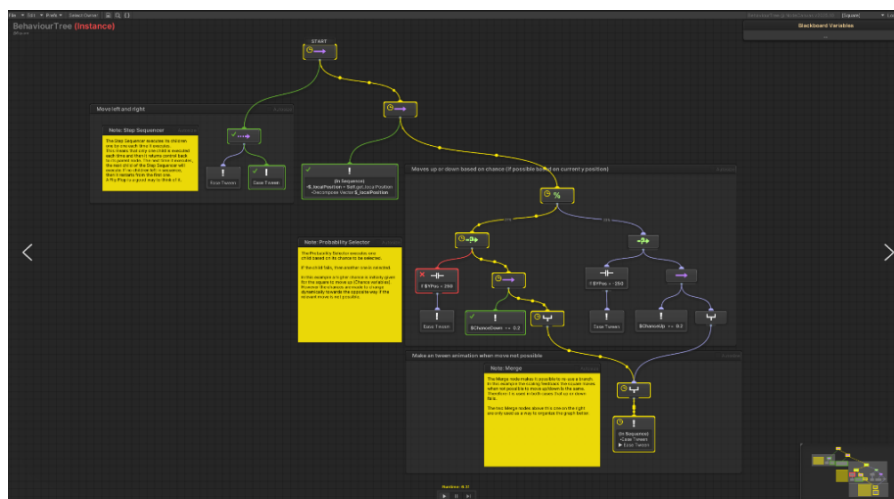
1990 년대 후반부터는 FSM(Finite State Machine)이 주요한 게임 AI 패러다임으로 자리잡기 시작했다. FSM은 유한한 상태 집합과 상태 간 전이 조건을 기반으로 동작하며, 각 상태에 고유한 행동을 부여하여 보다 구조적인 AI 동작을 가능하게 했다. 이 방식은 비교적 구현이 용이하고 디버깅이 간단하다는 장점이 있어 다양한 상용 게임에서 널리 사용되었다. 대표적인 예로는 플랫폼 게임의 적 캐릭터가 대기 상태에서 플레이어가 접근하면 추격하고, 일정 거리 내에 진입하면 공격하는 등의 전형적인 행동 패턴이 FSM 기반으로 구현되었다.

그러나 FSM은 상태 수와 전이 조건이 증가함에 따라 코드의 복잡성이 급격히 증가하는 '상태 폭발(State Explosion)' 문제를 야기한다. 새로운 행동을 추가하거나 조건을 세분화해야 할 경우, 기존의 상태 전이 로직 전체를 수정해야 하므로 유지보수가 어렵고 확장성이 낮다는 단점이 부각되었다. 이러한 문제를 해결하고자 등장한 것이 유틸리티 기반 AI, 행동 트리(Behavior Tree), GOAP(Goal-Oriented Action Planning), 머신 러닝 기반 AI 등이다.

행동 트리는 2000 년대 중반부터 AAA 게임 개발사들을 중심으로 채택되기 시작했으며, 복잡한 전략적 판단과 조건 분기를 간결하게 표현할 수 있다는 점에서 FSM을 대체하는 구조로 급부상했다. BT는 계층적 구조와 우선순위 기반의 선택 로직을 통해 유연한 행동 설계가 가능하며, 특히 협공, 피격 반응, 회피 행동 등 맥락 기반 행동을 구현하기에 적합하다. Unreal Engine, Unity 등 주요 게임 엔진에서도 BT를 기본 또는 확장 플러그인 형태로 지원하고 있으며, 이를 통해 디자이너와 프로그래머가 협업하여 직관적이고 강력한 AI 로직을 구축할 수 있다.



Unreal engine 에서의 bt 구조 사용



Unity engine 에서 Nodecanvas 를 사용한 bt 구조

최근에는 강화학습과 신경망을 활용한 AI가 주목받고 있으나, 이는 학습 비용이 높고 게임 기획에 제약이 있을 수 있다는 점에서 실시간 제어가 필요한 몬스터 AI에는 제한적으로 사용되고 있다. 따라서 본 논문에서는 산업 현장에서 실질적으로 널리 사용되는 FSM과 행동 트리를 중심으로 게임 AI의 발전 흐름을 분석하고, 두 구조의 특성과 실효성을 실험을 통해 비교하고자 한다.

2.2 FSM (Finite State Machine) 개요 및 적용 사례

FSM(Finite State Machine)은 게임 인공지능에서 가장 오랫동안 사용되어 온 제어 모델로, 제한된 개수의 상태(state)와 명확한 전이 조건(transition)을 기반으로 캐릭터의 행동을 제어한다. FSM의 기본 원리는 캐릭터가 특정 상태에 있으며, 외부 입력 또는 내부 조건이 충족되었을 때 정의된 전이 조건에 따라

다른 상태로 이동하는 방식이다. 각 상태는 하나의 행동을 나타내며, 상태 간의 흐름은 미리 정의된 조건에 의해 엄격하게 제어된다.

FSM의 가장 큰 장점은 그 구조가 단순하고 명확하다는 점이다. 상태 전이도와 행동 로직을 한 눈에 파악할 수 있으며, 개발 초기에는 빠르게 프로토타이핑하기에 적합하다. 예를 들어 적 몬스터가 Idle → Chase → Attack의 전투 루틴을 수행하거나, 플레이어의 위치에 따라 이동 경로를 변경하는 단순한 행동을 구성할 때 FSM은 효과적인 솔루션이 될 수 있다. Unity에서는 MonoBehaviour 기반 스크립트에 FSM 패턴을 구현하는 것이 일반적이며, Animator와 연동하여 캐릭터의 상태와 애니메이션을 연계하는 데에도 널리 활용된다. 또한 FSM은 플랫폼 구조(flat structure)를 기반으로 하기에 메모리 사용량이 낮고, 학습 곡선이 낮은 초급 개발자에게도 접근성이 뛰어나다.

그러나 FSM은 상태 수가 증가하거나, 전이 조건이 복잡해질 경우 구조적으로 불리한 상황에 직면하게 된다. 특히 게임 내 캐릭터의 행동이 늘어나고, 상황에 따라 조건 분기나 다중 전이가 필요할 경우 FSM은 "상태 폭발(state explosion)" 문제를 유발한다. 예컨대, Idle 상태에서 피격을 받아 도망가는 행동을 구현하려 할 때, 기존의 Idle → Chase → Attack 구조에 새로운 Flee 상태를 추가하면 Chase, Attack, Dead 등 기존 상태 모두에 대해 Flee로의 전이 조건을 고려해야 한다. 이는 상태 간 의존성을 증가시키고 코드 유지보수를 어렵게 만든다. 또한 상태 간 전이가 하드코딩되기 때문에 유연한 상황 대응이나 동적 행동 수정이 어렵다.

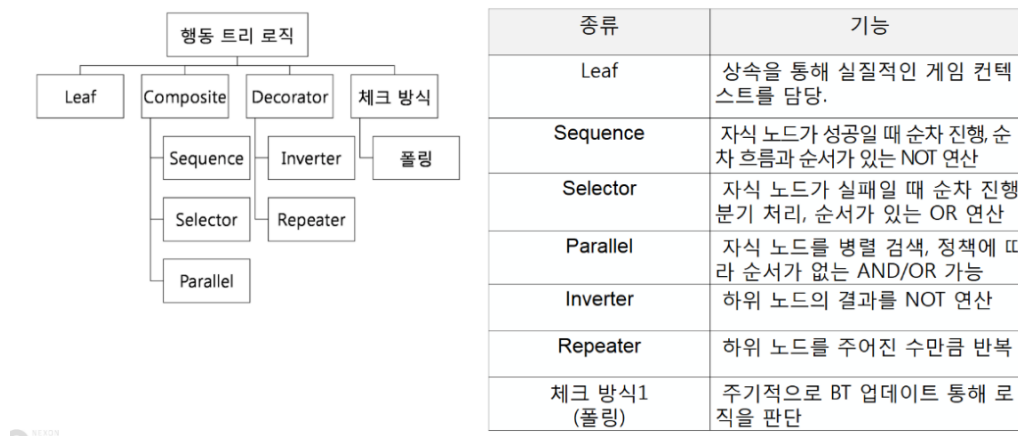
실제 상용 게임에서도 FSM의 한계는 자주 지적된다. 일정 수준 이상의 전략성과 맥락 기반 행동이 요구되는 게임에서는 FSM만으로 복잡한 전술 로직을 구현하기 어렵기 때문이다. 이로 인해 FSM은 단일 행동 루틴이나 보스 AI의 특정 페이즈 제어, 기본적인 순찰-추적 행동 등 비교적 단순한 AI에 적합하며, 복합적인 판단과 행동 우선순위 처리가 요구되는 경우에는 다른 구조(BT, GOAP 등)와의 결합이 필요하다. 그럼에도 불구하고 FSM은 그 단순함과 직관성으로 인해 여전히 많은 게임에서 중요한 역할을 담당하고 있으며, 본 논문에서도 FSM을 기반으로 한 몬스터 AI를 직접 구현하여 행동 트리 기반 AI와의 실험적 비교를 수행한다.

2.3 BT(Behavior Tree) 개요 및 적용사례

Behavior Tree(BT)는 복잡한 게임 시스템의 행동 흐름을 유연하게 제어하기 위해 개발된 계층적 구조의 AI 모델이다. 최근에는 상용 게임뿐 아니라 대규모 MMO 프로젝트에서도 활발히 채택되고 있으며, 대표적인 사례로 게임빌의 모바일 MMORPG 『로열 블러드』에서의 적용을 들 수 있다.

『로열 블러드』는 실시간으로 수많은 이벤트가 동시에 발생하는 구조를 지닌 모바일 MMORPG로, 그 특성상 수십 종의 전장 이벤트, 보스 스폰, 지역 점령전, 퀘스트 진행, 플레이어 간 상호작용 등을 하나의 이벤트 시스템 안에서 통합 관리해야 했다. 이에 따라 이벤트 흐름의 복잡성과 디버깅 난이도, 유지보수의 부담이 크게 증가하였으며, 기존의 스크립트 기반 이벤트 설계 방식으로는 시스템 확장이 어려운 한계에 봉착하게 되었다.

이러한 문제를 해결하기 위해 『로열 블러드』 개발팀은 서버 단 이벤트 처리의 핵심 구조로 Behavior Tree 아키텍처를 도입하였다. 공개된 사례에 따르면, 이 시스템은 수천 개의 BT 인스턴스가 동시에 실행되는 구조로 설계되었으며, 각 노드는 메시지 기반으로 동작하면서도 비동기 이벤트에 대응할 수 있도록 구성되어 있다. 예컨대, Selector 노드는 이벤트 우선순위에 따라 분기 처리를 수행하고, Sequence 노드는 전장 상태 확인, 몬스터 스폰, 알림 브로드캐스트 등 다단계 처리를 순차적으로 실행한다. Decorator는 조건에 따라 노드 실행 여부를 결정하며, Leaf 노드는 실제 액션 또는 조건 판단을 담당한다. 모든 BT 구성은 XML 기반으로 기술되어 있으며, 이를 통해 디자이너와 기획자 또한 직접 노드 구조를 설계 및 수정할 수 있도록 지원된다.



『로열 블러드』에서 실제 도입된 행동 트리 로직 (BT 구조)

이러한 구조는 반복적으로 발생하는 복합 이벤트를 모듈 단위로 분리하여 재사용 가능하게 하며, 로직 간의 의존도를 낮춰 상태 전이의 명확성과 시스템 안정성을 확보할 수 있도록 한다. 또한 트리 기반 구조는 이벤트 흐름을 시각적으로 표현할 수 있어, 디버깅과 유지보수 과정에서도 뛰어난 가시성을 제공한다. 실제

사례에서도 신규 콘텐츠 추가 시 전체 구조를 수정하지 않고 노드 단위로 삽입 및 제거가 가능해 개발 효율성과 확장성이 크게 향상되었다고 보고되었다.



『로열 블러드』 이벤트 기능에서 실제 도입된 행동 트리 로직 (BT 구조)

본 연구에서도 이러한 사례를 참고하여 FSM 기반의 단일 흐름 구조에 한계를 느끼던 몬스터 AI, 무한맵 구성, 웨이브 생성 로직 등에 Behavior Tree 구조를 도입하고자 하였다. 특히 복수의 조건을 기반으로 하는 동적 전환, 이벤트 간 우선순위 조정, 행동 모듈의 재사용 가능성을 중심으로 실험을 설계하였다. 이러한 구조적 접근은 학부 수준의 프로젝트에서도 설계의 일관성을 유지하며, 테스트와 유지보수의 부담을 낮출 수 있는 실질적 장점을 제공하였다.

『로열 블러드』의 사례는 Behavior Tree가 단순한 행동 제어를 넘어서 복합 시스템의 핵심 아키텍처로도 충분히 기능할 수 있음을 보여주며, 본 논문은 이를 기반으로 BT 구조가 다양한 규모와 장르의 게임 개발 환경에서 실용적 대안이 될 수 있음을 제안한다.

종합 평가

항목	상세 목표	평가
도입 비용	검토를 포함한 구현 기간은 최대 2주 이내	부분 달성
디버깅 편의	디버깅에 용이해야 한다.	달성
변경 용이성	기획적 실험과 탐구에 대응할 수 있어야 한다.	달성
데이터 생산성	데이터 드리븐 설계에 대응해야 한다.	부분 달성
확장성	콘텐츠 로직의 확장이 용이해야 한다.	달성
호환성	다른 모듈과 결합할 수 있어야 한다.	달성
안전성	구조적 버그 발생이 없어야 한다.	달성
관리 편의성	협업에 용이한 구조여야 한다.	부분 달성
올바른 동작	제 기능을 할 수 있어야 한다.	달성

NDC

67/70

『로열 블러드』에서 공개한 BT 구조를 도입했을때의 최종 결과

2.4 BT(Behavior Tree) 구조 도구의 등장과 확산

게임 AI 설계 방식이 복잡성과 확장성 측면에서 더욱 정교한 제어를 요구하게 되면서, Behavior Tree(BT)는 기존의 Finite State Machine(FSM) 구조를 대체할

수 있는 대안으로 자리잡기 시작하였다. 초기에는 AAA 게임 개발사를 중심으로 도입되던 BT 구조는, 그 유연성과 시각적 가독성, 유지보수 용이성 등의 장점으로 인해 점차 다양한 장르와 규모의 게임 개발사들에 의해 채택되고 있다.

이러한 구조적 전환과 더불어 Unity, Unreal Engine 등 주요 게임 엔진에서도 BT 설계와 실행을 지원하는 다양한 도구들이 등장하였다. 특히 Unity 환경에서는 시각적 편집과 실행이 가능한 상용 도구 모음들이 활발히 사용되고 있으며, 대표적인 예로 Panda BT, NodeCanvas, Behavior Designer 등이 있다.

Panda BT는 경량화된 DSL(Domain Specific Language)을 사용하여 코드 중심으로 행동 트리를 설계할 수 있는 툴로, 고정된 에디터보다는 스크립트 기반 제어를 선호하는 개발자들에게 적합하다. 심플한 구문 구조와 빠른 실행 환경이 특징이며, 디버깅 또한 텍스트 기반 로그로 간결하게 수행된다.

NodeCanvas는 FSM과 BT를 모두 지원하는 종합 시각화 툴로, 복잡한 캐릭터 행동과 상태 흐름을 하나의 통합 그래프 기반 인터페이스로 구성할 수 있다는 점에서 중대형 프로젝트에 적합하다. 특히 BT 외에도 대화 시스템(Dialogue Tree), 퀘스트 트리 등 확장 모듈과의 연동이 가능하여 유연한 게임 시스템 설계에 활용된다.

Behavior Designer는 시각적 노드 기반 인터페이스와 함께 다양한 애드온을 제공하여, Unity의 애니메이션 시스템 및 내비게이션 기능과의 통합이 용이하다. 위 그림에서는 그 중 하나인 Movement Pack이 예시로 나타나 있으며, 이는 이동, 회피, 추적 등과 같은 캐릭터 행동을 손쉽게 구성할 수 있도록 미리 정의된 노드 집합을 포함한다.

결과적으로 Behavior Tree 구조는 단순한 설계 기법의 대안을 넘어, 게임 AI 분야에서 체계적이고 확장 가능한 행동 제어의 핵심 패러다임으로 자리매김하고 있다. 특히 BT는 단순한 상태 전이 구현에 그치지 않고, 실시간 전략 게임의 유닛 제어, 복합 퀘스트 처리, 이벤트 기반 전투 로직, 협동형 NPC의 상황 인식 및 판단 등 다양한 고도화된 AI 설계에 효과적으로 활용되고 있다. 이러한 구조는 단순한 반응 기반 AI를 넘어서, 문맥 인식 기반의 동적 행동 설계를 가능하게 하며, 플레이어 경험의 몰입도와 상호작용의 정교함을 실질적으로 향상시키는 데 기여한다.

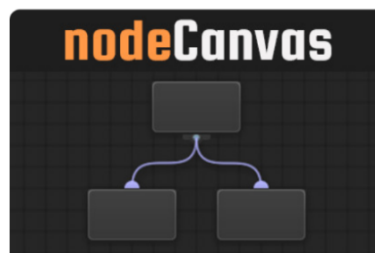
본 논문에서는 이러한 도구들을 실제 Unity 기반의 게임 시스템에 적용하고, 각 구조의 설계 효율성과 실행 성능, 유지보수 편의성 등을 다각적으로 분석함으로써 Behavior Tree 구조의 실용성과 확장 가능성에 대한 실증적 근거를 제시하고자 한다.



Panda BT 2

★★★★★ 5 (13)
ERIC BEGUE

\$85.00



NodeCanvas

★★★★☆ 4.7 (363)
PARADOX NOTION

\$120.00



Behavior Designer - Movement Pack

★★★★☆ 4.7 (196)
OPSIVE

\$15.00

제 3 장 시스템 아키텍처 및 구현 방법

3.1 개발 환경 및 방법론

뱀파이어 서바이벌류 게임은 로그라이크(Roguelike)와 액션 슈팅 장르의 하위 분류로, 플레이어가 끊임없이 몰려오는 적을 상대로 생존을 목표로 하는 실시간 전투 중심의 게임이다. 일반적으로는 자동 전투 또는 반복 공격 방식이 특징이며, 시간 경과에 따라 점점 더 강력한 적이 등장하고, 플레이어는 레벨업을 통해 무기나 능력을 강화하여 생존 시간을 연장해 나간다. 이러한 장르는 고난도의 전략적 판단이 요구되며, 다양한 조건 기반 시스템과 상태 전환 로직이 유기적으로 작동하는 특징을 가진다.

본 연구는 Unity 엔진(2021.3 LTS)을 기반으로 직접 제작한 2D 뱀파이어 서바이벌류 액션 게임을 실험 환경으로 활용하여, FSM(Finite State Machine)과 Behavior Tree(BT) 기반의 AI 시스템 구조를 비교·분석하였다. Unity는 게임 로직 구현의 유연성, 시각적 디버깅의 용이성, 다양한 외부 라이브러리와 플러그인의 통합 가능성 등 실험적 개발 환경에 매우 적합한 조건을 제공한다. 초기 개발 단계에서는 게임 내 모든 로직을 FSM 기반으로 일괄 구현하였다.

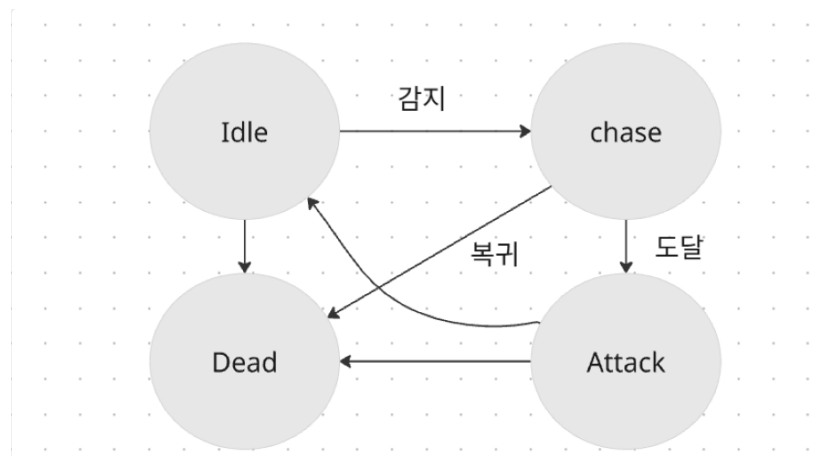
이는 게임 내 AI, 맵 구성, 상태 전이, 레벨 기반 무기 전환 등 다양한 기능이 FSM 구조 하에 동작하도록 한 것이다. FSM은 각 상태와 그 전이 조건을 명확히 정의함으로써 전체 로직 흐름을 체계적으로 구성할 수 있는 장점이 있다. 그러나 개발이 진행될수록 시스템의 복잡도가 증가함에 따라 FSM 구조는 확장성과 유지보수 측면에서 점차 한계를 드러냈다.

따라서 후속 단계에서는 기존 FSM 구조를 유지한 상태에서 핵심 알고리즘을 Behavior Tree(BT) 구조로 점진적으로 리팩토링하였다. 특히 몬스터 AI, 무한맵 구성 로직, 플레이어 무기 전환 시스템 등 반복적인 판단과 조건 분기가 핵심인 모듈을 중심으로 BT 구조를 도입하였으며, FSM과 BT 구조 간 성능과 설계 효율성을 실험적으로 비교·분석하였다. 이로써 본 연구는 단일 기능 비교를 넘어, 게임 시스템 전반에서 FSM과 BT의 실질적인 차이를 평가하고자 하였다.

3.2 FSM 구조로의 기본 게임 구현

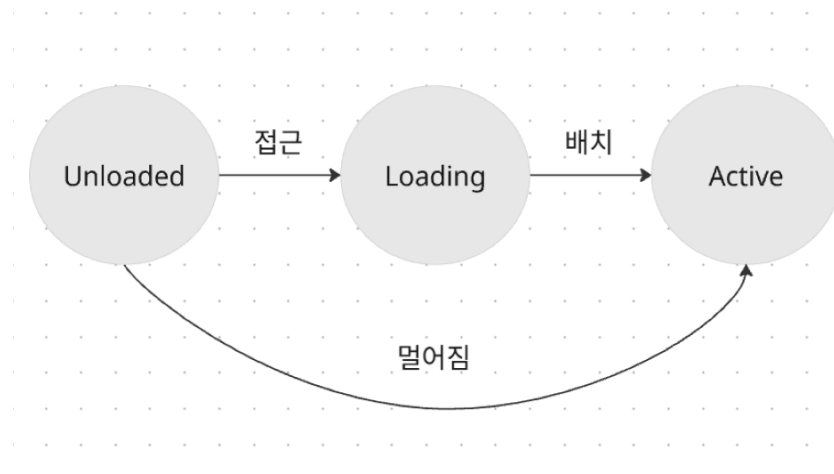
FSM(Finite State Machine) 기반 구조는 각 기능을 상태와 전이 조건으로 나누어, 순차적이고 조건 중심의 로직 제어를 통해 게임을 구성하는 방식이다. FSM은 단순한 제어 흐름에서는 설계와 구현이 용이하며, 초기에 빠른 프로토타입 제작에 적합하다. 본 연구에서는 다음과 같은 시스템들을 FSM 구조로 설계하였다.

(1) 몬스터 AI FSM



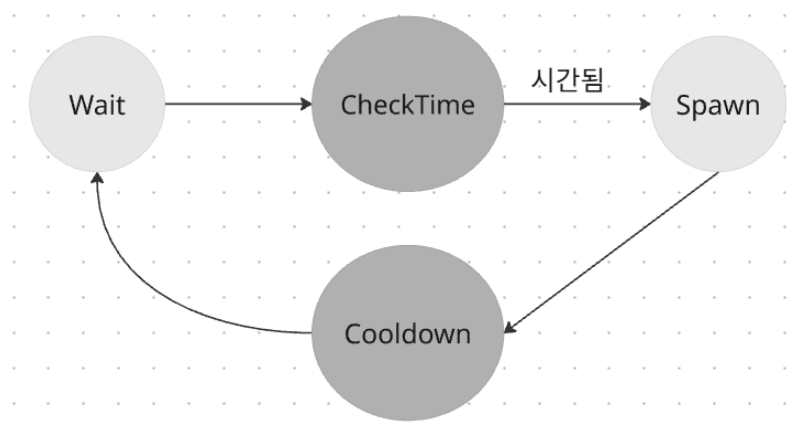
몬스터는 Idle(대기), Chase(추격), Attack(공격), Dead(사망)의 네 가지 상태를 가진다. 플레이어가 감지되면 Idle에서 Chase로, 공격 범위에 도달하면 Attack으로, 체력이 0이 되면 Dead 상태로 전이된다. 이러한 상태 전이는 조건문을 기반으로 Enemy.cs 스크립트 내에서 직접 제어되며, 각 개체별 FSM 인스턴스가 독립적으로 운용된다.

(2) 무한맵 FSM



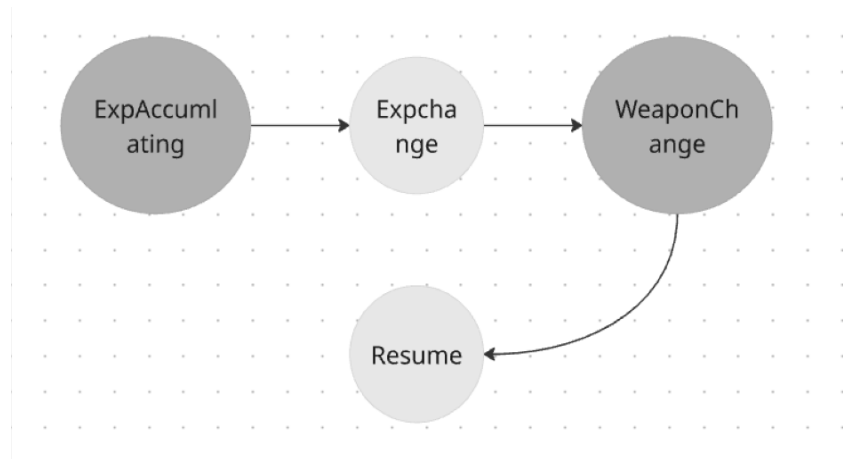
플레이어가 화면의 경계에 도달했을 때 새로운 맵 타일을 생성하거나 기존 타일을 재배포하는 방식으로 무한맵이 구현된다. 이때 타일은 Unloaded, Loading, Active 등의 상태를 가지며, FSM을 통해 상태 전이가 이루어진다. 각 타일은 재사용 가능한 오브젝트로 설정되며, FSM의 상태 전이를 통해 활성화 여부와 위치를 관리한다.

(3) 웨이브 생성 FSM



일정 시간 간격으로 몬스터 웨이브를 생성하는 기능은 Wait → CheckTime → Spawn → Cooldown 흐름의 FSM으로 설계되었다. 일정 시간 대기 후 시간 조건이 만족되면 새로운 몬스터 집단이 생성되고, 이후 쿨다운 상태로 전환되어 다음 웨이브 생성까지의 대기 시간이 주어진다. 웨이브의 단위 구성은 고정되어 있으며, 난이도에 따른 동적 조정은 별도로 구현되지 않았다.

(4) 플레이어 무기 전환 FSM



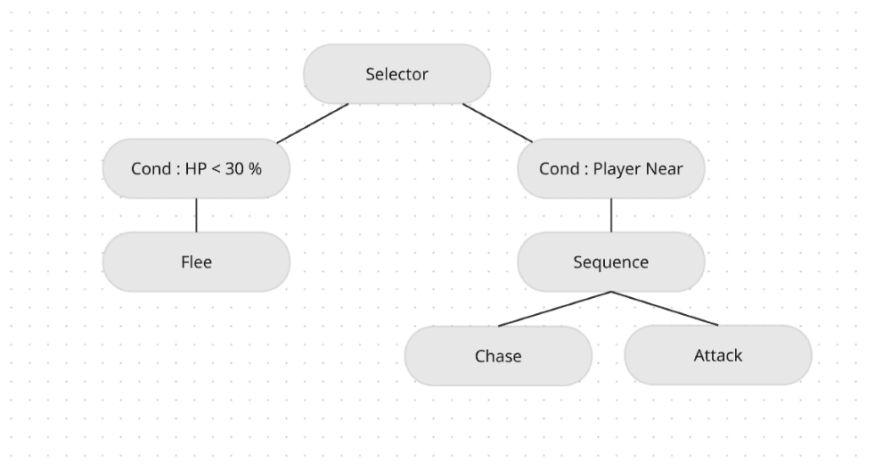
플레이어는 적을 처치하거나 시간 경과를 통해 경험치를 획득하고, 일정 수치를 넘어서면 레벨업한다. 이때 무기가 자동으로 변경되며, 이는 ExpAccumulating → LevelCheck → WeaponChange → Resume의 상태 흐름으로 구현되었다. FSM 내에서는 경험치 수치가 일정 기준에 도달했는지를 판단하여 전이 조건을 만족시킬 경우 무기 오브젝트를 교체하고, 새로운 애니메이션이나 타격 효과를 적용한다.

FSM 구조는 상태 기반 판단이 명확하고 구조적이지만, 조건이 복잡해지거나 새로운 기능이 추가될 때마다 기존 구조 전체에 영향을 주어 확장성과 유지보수 측면에서 부담이 증가하는 단점이 존재한다.

3.3 BT 기반 구조로의 알고리즘 구현

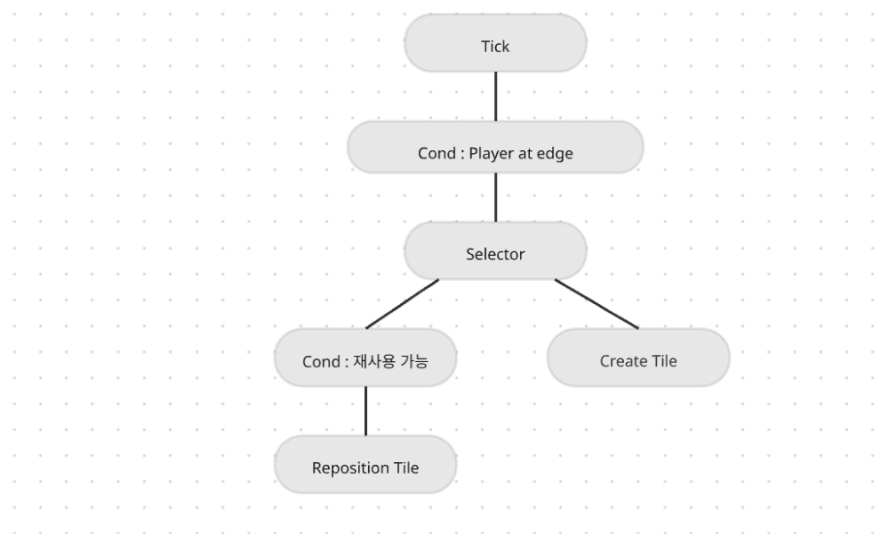
FSM 구조로 완성된 게임의 핵심 기능 중 반복적인 조건 분기 및 상황 판단이 많은 모듈에 대해 Behavior Tree(BT) 기반 구조를 도입하였다. BT는 계층적 노드 기반 로직으로, 각 조건과 행동을 모듈화할 수 있어 복잡한 판단 구조에 유리하다. 본 연구에서는 다음 세 가지 시스템에 대해 BT 구조를 적용하였다.

(1) 몬스터 AI BT



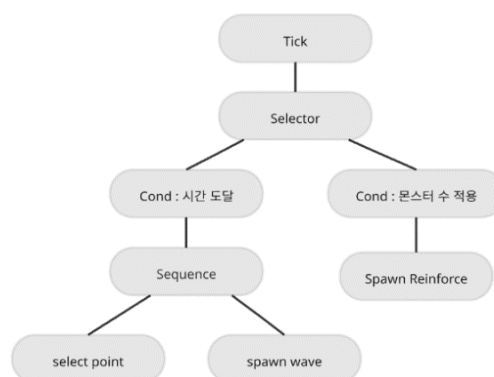
몬스터는 FSM 구조보다 더 세분화된 행동을 BT 기반으로 표현할 수 있었다. Condition 노드에서 플레이어 거리, 체력, 장애물 존재 여부 등을 판단하고, Selector 또는 Sequence 노드를 통해 추격, 회피, 공격 등의 행동을 선택한다. 각 Action 노드는 실제 이동, 애니메이션 재생, 데미지 계산 등 게임 내 물리적 효과를 수행한다. 이러한 구조는 각 행동을 독립적으로 모듈화할 수 있어, 유지보수성과 확장성이 FSM에 비해 매우 우수했다.

(2) 무한맵 BT



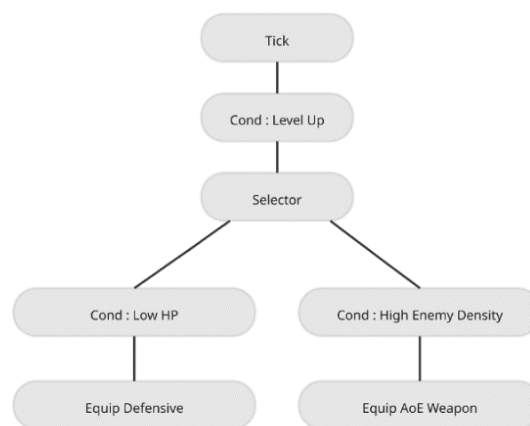
무한맵 시스템에서는 플레이어가 경계에 도달할 경우 Condition 노드가 이를 감지하고, 기존 타일의 재사용 가능 여부에 따라 Selector 노드에서 분기된다. 재사용이 가능하면 해당 타일을 재배치하고, 불가능하면 새로운 타일을 생성하는 Sequence가 실행된다. Decorator를 활용하여 조건 만족 여부에 따라 행동이 실행되며, 복잡한 타일 배치 조건과 우선순위가 적용되었을 경우에도 로직을 유연하게 구성할 수 있었다.

(3) 플레이어 무기 전환 BT



플레이어의 레벨업 조건이 충족되었는지를 Condition 노드에서 판단한 뒤, Selector를 통해 현재 보유 무기, 적 밀도, 체력 상태 등을 종합적으로 평가한다. 이를 바탕으로 가장 적합한 무기로 전환하는 Action 노드가 실행되며, UI 갱신 및 시각적 이펙트도 동시에 트리 내에서 처리된다. 기존 FSM 구조에서는 단일 조건만을 전제로 설계되었다면, BT 구조에서는 다양한 조건을 동시에 고려하는 전략적 무기 전환이 가능하였다.

(4) 웨이브 생성 BT



BT 구조의 웨이브 생성 시스템은 FSM 보다 조건 제어의 유연성과 분기 관리의 명확성을 확보할 수 있었다. Condition 노드에서는 시간 경과, 현재 필드에 존재하는 몬스터 수, 플레이어 생존 여부 등 복수의 조건을 평가한다. 이후 Selector 노드를 통해 웨이브 생성 여부를 판단하고, Sequence 노드를 통해 적절한 수의 몬스터를 스폰 포인트에 배치하는 Action 노드들이 실행된다. 웨이브 간 간격 제어, 특정 조건 만족 시 강화형 웨이브 등장 등 복합 조건을 적용하기 용이하며, 전반적인 게임 흐름을 더 정교하게 통제할 수 있었다.

이처럼 BT 구조는 각 게임 시스템의 기능을 세부 단위로 나누어 구성할 수 있기 때문에, FSM 구조에 비해 조건 변화에 따른 유연한 대처와 구조적 일관성을 유지할 수 있는 장점이 있었다. 특히 실험 결과에서도 보여지듯, BT 구조는 복잡한 판단이 요구되는 실시간 게임 시스템에 더욱 적합한 알고리즘 구조임이 입증되었다.

3.4 실험 시나리오 및 테스트 항목 정의

본 실험은 FSM 기반 구조와 BT 기반 구조를 동일한 게임 환경에서 구현한 후, 기능별 성능과 동작 특성을 비교 분석하는 방식으로 설계되었다. Unity 엔진 내

로그 수집 기능과 개발자 도구를 활용하여 각 구조의 성능 지표를 수집하였으며, 반복 가능한 테스트 시나리오를 기반으로 정량적·정성적 분석을 병행하였다.

정량적 비교 항목		정성적 비교 항목	
항목	항목	몰입도	실험 참가자가 체감한 AI의 몰입도 수준에 대한 설문 평가
반응속	특정 조건 발생 후 첫 행동까지 걸린 시간	자연스러움	외부 평가자가 판단한 AI 행동의 현실성 및 동작 자연스러움
오류율	예외 조건에서의 충돌, 무응답 등	유지보수성	구조 확장 및 기능 추가 시 개발자의 난이도 체감 정도 평가
전환 효율성	상태 또는 행동 간의 전이 소요 시간		
행동 다양성	고유한 행동 분기 또는 패턴 수		

4 실험결과 및 분석

4.1 정량적 분석 결과

FSM과 BT 기반 몬스터 AI 구조의 성능을 비교하기 위해 본 연구는 반응 속도, 오류율, 전환 효율성, 행동 다양성의 네 가지 항목을 기준으로 정량적 실험을 수행하였다. 이 실험은 동일한 환경에서 구조별로 총 5회씩 반복 수행되었으며, 평균값과 표준편차를 기반으로 분석되었다.

첫째, 반응 속도는 플레이어가 몬스터의 탐지 범위에 진입한 시점과 몬스터가 첫 반응(추격, 도주 등)을 개시하는 시점 사이의 시간 차로 측정하였다. Unity의 Time.time API를 활용하여 해당 시점들을 로깅하였으며, FSM은 평균 140ms, BT는 평균 110ms의 반응 속도를 기록하였다. BT 구조는 조건 평가와 행동 실행이 트리 구조로 모듈화되어 있어 조건 탐색 시간이 짧고 빠르게 반응할 수 있었던 반면, FSM은 상태 전이 조건을 일일이 탐색해야 하므로 반응이 느려지는 경향을 보였다.

둘째, 오류율은 몬스터가 실행해야 할 행동을 제대로 수행하지 못하고 멈추거나 불완전하게 전이되는 경우를 비율로 나타낸 지표이다. FSM에서는 상태 간 전이가 복잡해지면서 충돌이나 조건 누락으로 인해 평균 7.4%의 오류가 발생하였으며, 반면 BT 구조는 조건 노드 간 독립성이 높고 우선순위 기반 실행이 가능하여 평균 오류율 2.1%로 안정성이 높게 나타났다.

셋째, 전환 효율성은 상태 또는 행동 노드 간의 전환 시점 사이의 시간, 불필요한 중복 조건 평가 횟수 등을 종합한 평가 항목이다. FSM 구조는 평균 전환 시간

85ms를 기록하며 다수의 중첩 조건 탐색이 발생하였으나, BT 구조는 평균 45ms로 효율적인 전환이 이루어졌고 조건 중복 발생률도 현저히 낮았다. 이는 BT의 시퀀스/셀렉터 노드가 조건의 우선순위와 흐름을 명확히 제어하기 때문이다.

넷째, 행동 다양성은 일정 시간(1분) 동안 몬스터가 수행한 고유한 행동 개수로 측정되었다. FSM 구조는 평균 3.2개의 반복 행동을 보인 반면, BT는 평균 4.6개의 고유 행동을 보여 더 다양한 전략적 행동이 가능함을 나타냈다. 이는 트리 노드를 통해 새로운 행동 분기를 쉽게 삽입할 수 있다는 구조적 이점에서 비롯된다.

결론적으로, BT 기반 몬스터 AI는 반응성과 안정성, 효율성, 행동 다양성 면에서 FSM 구조보다 전반적으로 우수한 성능을 보였으며, 특히 복잡한 행동 흐름이 요구되는 게임 상황에서 더욱 강력한 확장성과 유연성을 제공함을 실험적으로 입증하였다.

4.2 정성적 분석 결과

정성적 분석은 FSM과 BT 기반 AI의 주관적 체감 차이를 파악하기 위해 피험자 10명을 대상으로 설문 조사를 통해 수행되었다. 각각의 실험 참가자는 두 구조의 게임을 5분간 플레이한 후, Likert 5점 척도 기반의 설문 문항에 응답하였다. 본 연구는 '몰입도', '자연스러움', '유지 보수성' 세 항목에 초점을 맞춰 구조 간의 질적 차이를 분석하였다.

첫째, 몰입도는 피험자가 게임 중 얼마나 집중하고 AI의 행동 흐름에 몰입할 수 있었는지를 평가하는 항목으로, BT 구조의 평균 점수는 4.4점, FSM은 3.1점을 기록하였다. BT 구조는 도주, 협공 등의 동적인 행동이 자연스럽게 연결되며 전개되기 때문에 플레이어가 AI의 반응을 보다 직관적으로 인지하고 상호작용에 깊게 몰입할 수 있었다. 반면 FSM은 반복적인 패턴으로 인해 몰입도가 상대적으로 낮게 평가되었다.

둘째, 자연스러움은 몬스터의 반응과 움직임이 인간의 기대와 얼마나 일치하는지를 측정한 항목이다. BT 구조는 조건 기반의 트리 흐름 덕분에 예상치 못한 행동이나 복잡한 전술 행동(예: 도주 후 재공격, 협공 등)이 부드럽게 구현되어 평균 4.5점을 획득하였다. FSM은 행동 전환 시 끊김이 있거나, 단순한 행동 반복이 많아 2.8점으로 낮은 평가를 받았다.

셋째, 유지 보수성은 피험자 중 개발 경험이 있는 참여자들이 코드 구조의 가독성과 확장성에 대해 평가한 항목으로, BT 구조는 행동 노드 단위로 모듈화되어 있어 수정 및 추가가 용이하다는 점에서 높은 평가를 받았다. 설문에서는 “BT는 새로운 행동을 추가할 때 전체 구조를 건드릴 필요가 없다”는 응답이 다수였으며, FSM은 조건문 중첩과 상태 간 결합도가 높아 구조 변경 시 리스크가 크다는 지적이 있었다.

종합적으로, 정성적 평가 결과 BT 구조는 몰입도, 행동의 자연스러움, 유지보수의 용이성 측면에서 FSM 구조보다 긍정적인 반응을 얻었으며, 이는 BT 기반 설계가 사용자 체감 경험뿐 아니라 개발 효율성 측면에서도 우위를 점할 수 있음을 시사한다.

정성적 분석에서는 BT 기반 구조에 대해 높은 평가가 주어졌다. 특히 '예상치 못한 협공'이나 '위기 시 도주 반응'과 같은 전략적 요소에서 플레이어는 명확한 차이를 체감했으며, "AI가 살아 있는 것 같았다"는 주관식 응답도 다수 포함되었다. FSM 구조는 "반응이 단조롭고 반복적이었다"는 평가가 주를 이루었다

4.3 FSM과 BT의 성능 비교 종합

FSM과 BT 구조는 모두 게임 AI 개발에 널리 사용되는 방식이지만, 본 연구를 통해 이 두 구조는 정량적 성능과 정성적 체감에서 분명한 차이를 보임을 확인할 수 있었다. 실험 결과 BT 구조는 정량적으로 더 빠른 반응 속도, 낮은 오류율, 효율적인 전환 시간, 그리고 높은 행동 다양성을 기록하였다. 이는 BT 구조가 조건 기반 흐름을 계층적으로 처리함으로써 복잡한 로직을 보다 안정적이고 유연하게 구성할 수 있음을 의미한다.

정성적 평가에서도 BT 구조는 피험자들에게 더 높은 몰입도와 행동의 자연스러움을 제공했으며, 특히 개발자 경험을 가진 피험자들에게는 구조의 명료성과 유지보수 용이성 측면에서도 높은 점수를 받았다. 반면 FSM 구조는 행동 수가 증가할수록 상태 간 전이가 복잡해지고 코드의 결합도가 높아져 수정이 어렵다는 단점이 뚜렷하게 드러났다.

FSM은 설계가 간단하고 직관적이기 때문에 초기 구현에는 유리하나, 행동 단위가 늘어날수록 전이 조건이 비대해지고 반복적 로직이 많아지는 경향이 있다. 이와 대조적으로 BT는 각 행동이 독립적 노드로 구성되어 있어 모듈 단위로

관리 가능하며, 트리의 확장만으로 새로운 행동을 쉽게 삽입할 수 있어 확장성과 유지보수성이 뛰어나다.

결론적으로, 단순한 AI에는 FSM도 충분한 기능을 제공할 수 있으나, 협공, 도주, 탐지 등과 같이 복합적인 조건과 전략적 전개가 요구되는 현대 게임 AI에는 BT 구조가 더 적합하다. 특히 본 연구와 같이 실제 게임 플레이를 통한 실험에서는 이러한 구조적 차이가 사용자 체감에도 영향을 주었으며, 이는 BT 구조가 상업적 게임 개발에서도 유리한 선택이 될 수 있음을 시사한다.

5 결론

5.1 연구 요약 및 결론

본 연구는 Unity 기반의 2D 뱀파이어 서바이벌류 게임을 실험 환경으로 활용하여, 게임 내 AI 및 시스템 구현에 사용되는 두 가지 대표적 구조인 FSM(Finite State Machine)과 BT(Behavior Tree)의 성능 및 구조적 특성을 비교·분석하였다. 실험 대상 시스템은 몬스터 AI, 무한맵 구성, 플레이어 무기 전환, 웨이브 생성 시스템 등으로 구성되었으며, 동일한 시나리오 하에 두 구조를 각각 적용하였다.

FSM 구조는 명확한 상태 전이 기반의 설계로 인해 구현이 직관적이고 단순한 로직 제어에는 적합한 구조임이 확인되었다. 그러나 시스템이 복잡해질수록 상태 수와 전이 조건이 기하급수적으로 증가하며, 확장성과 유지보수 측면에서 한계가 존재하였다. 이에 반해 BT 구조는 조건 분기와 행동 흐름을 모듈화하여 구성함으로써 유연한 판단 구조와 높은 재사용성을 제공하였으며, 복잡한 전략적 로직을 효과적으로 구현할 수 있었다.

정량적 실험 결과, BT 구조는 반응 속도, 오류율, 행동 다양성 등에서 FSM 대비 우수한 결과를 보였고, 정성적 평가에서도 몰입도, 자연스러움, 유지보수성 측면에서 긍정적인 반응을 얻었다. 이를 통해 BT 구조는 게임 내 알고리즘 구조 설계에 있어 보다 효과적인 대안이 될 수 있음을 실증적으로 입증하였다.

5.2 향후 연구 방향

본 연구는 FSM과 BT 구조의 비교를 통해 게임 내 알고리즘 설계에 대한 기초적인 방향성을 제시하였으나, 실험 대상이 제한적인 특정 장르와 구조에

국한되어 있었다는 점에서 한계가 존재한다. 향후 연구에서는 다음과 같은 확장 방향이 고려될 수 있다.

첫째, 다양한 게임 장르(예: RPG, 전략 시뮬레이션, 퍼즐 등)에 BT 구조를 적용하여 구조적 특성과 성능을 종합적으로 분석하는 다장르 비교 연구가 필요하다.

둘째, 현재 연구에서는 정적인 트리 구조 기반의 BT를 적용하였으나, 향후에는 머신러닝 기반의 행동 트리 생성, 조건 노드 최적화, 행동 우선순위 학습 등 AI 학습 요소를 결합한 동적 트리 구조에 대한 연구가 필요하다.

셋째, 게임 내 플레이어 행동 로그를 기반으로 한 AI 자동 조정 시스템, 혹은 플레이어 적응형 난이도 조절 알고리즘과 같은 고도화된 BT 응용 사례에 대한 실험도 고려할 수 있다.

넷째, FSM과 BT의 하이브리드 구조를 설계하여 상황에 따라 구조를 전환하거나 병행 적용하는 복합형 알고리즘 구조의 설계 및 실증적 효과 분석도 중요한 후속 과제가 될 수 있다.

이러한 향후 연구를 통해 게임 알고리즘 구조의 효율성과 유연성은 물론, 플레이어 경험을 향상시키는 정교한 알고리즘 구조 설계가 가능할 것으로 기대된다.

3 참고문헌

- [1] [한국컴퓨터정보학회 2016년도 제53차 동계학술대회논문집 24권1호, 2016 Jan. 12,](#)
2016 년
- [2] [인터넷정보학회논문지 = Journal of Korean Society for Internet Information, v.8 no.6,](#)
2007 년
- [3] A study on The Implementation of Monster AI using Finite-State Machine
Jae-Won Jo O , Jung-Won Bang
- [4] Kim, J., & Choi, S. (2020). *Designing Enemy AI in Unity using Finite State Machines*. Journal of Game Development Research, 15(2)
- [5] Lee, H., & Park, Y. (2021). *A Study on Maintainability of FSM-based AI in Complex Game Systems*. KIISE, HCI2021
- [6] GOBT: A Synergistic Approach to Game AI Using Goal-Oriented and Utility-Based Planning in Behavior Trees
저자: Yoosung Hong, Tianhao Yan, Jinseok Seo Epic Games. (2018).
- [7] Unity Technologies. (2020). *GDC Talk: Behavior Trees and Modular AI Design in Unity*.
<https://unity.com/>
- [8] DEVELOPMENT OF GAME AI USING BEHAVIOR TREE : BASED ON THE UNITY5 ENGINE Jun-O Choi (KwangWoon University)Hyun-Joo Park (SangMyung University), 2018
- [9] [NDC Replay](#)